

Reorganizing the ISDF Grid by Crystal Symmetry

A step-by-step, plain-language guide from the (k, I) basis to symmetry labels

July 30, 2026

What this note is about

In the FFT-ISDF / THC representation of the electron repulsion integrals, everything is built from two objects living on an *interpolation grid*: the interpolation vectors X (`inpv_kpt`) and the central Coulomb tensor W (`coul_kpt`). Their natural index is (k, I) : a crystal momentum k and an interpolation point I .

This note explains, in plain language and one small step at a time, **how the crystal’s space-group symmetry lets you relabel that (k, I) basis into “symmetry-adapted” labels**, and why doing so makes a symmetric object like W collapse to a tiny amount of data. It is a conceptual / intuition-building guide, not a derivation. The running example throughout is **diamond on a $2 \times 2 \times 2$ k -mesh**, where there are $n_k = 8$ momenta and $n_{\text{IP}} = 136$ interpolation points (so $8 \times 136 = 1088$ basis states in all), and the crystal has 48 rotation/reflection symmetries.

The starting point. We assume we are already in the (k, I) basis. Getting there is the Fourier transform over lattice cells, which “uses up” the translational symmetry and makes k a good label. What remains to organize is the **point group** — the 48 rotations and reflections — which does two things: it moves the momenta k around, and it shuffles the interpolation points I . The five steps below are how you fold that remaining symmetry into the labels.

Step 1 — Group the k -points into families

Bring in the crystal’s rotations and reflections. Each one, applied to a momentum k , moves it to *another* momentum (rotating the arrow k). Take one k -point and hit it with all the rotations/reflections: you get a **family of k -points that get shuffled among each other**. The 8 momenta of diamond $2 \times 2 \times 2$ fall into **3** families:

- **Family A:** just $(0, 0, 0)$ — the center; every rotation leaves it where it is, so it is alone.
- **Family B (4 points):** $(0, 0, \frac{1}{2}), (0, \frac{1}{2}, 0), (\frac{1}{2}, 0, 0), (\frac{1}{2}, \frac{1}{2}, \frac{1}{2})$ — rotations shuffle these four into each other.
- **Family C (3 points):** $(0, \frac{1}{2}, \frac{1}{2}), (\frac{1}{2}, 0, \frac{1}{2}), (\frac{1}{2}, \frac{1}{2}, 0)$.

($1 + 4 + 3 = 8$.) Within a family, the rotations only carry you from one member to another.

Step 2 — At one k per family, keep the operations that leave it fixed

Pick one representative from each family. Of the 48 operations, split them (for that specific k) into the ones that *move* it to another family member and the ones that **leave it exactly in place**. Keep only the “leave-it-fixed” ones. Counting them:

- $(0, 0, 0)$: it is the center, so **all** 48 leave it fixed.
- $(0, 0, \frac{1}{2})$: only 12 leave it fixed; the other 36 move it to its 3 family-mates.
- $(0, \frac{1}{2}, \frac{1}{2})$: 16 leave it fixed.

Tidy bookkeeping: $(\text{family size}) \times (\text{number that leave it fixed}) = 48$ every time $(1 \times 48, 4 \times 12, 3 \times 16)$ — the operations spread the point evenly across its family.

Step 3 — Reorganize that k -point’s 136 interpolation points into simple patterns

At the representative k -point you have its 136 interpolation points, and the “leave-it-fixed” operations from Step 2 shuffle those points among themselves. Now build **combinations** (weighted sums) of the 136 points that behave in the *simplest possible ways* under those operations:

- **size-1 pattern:** a combination that each operation just multiplies by a number (a phase); it never mixes with anything else.
- **size-2 pattern:** a pair of combinations the operations only rotate into each other.
- likewise size-3, etc.

Sort all 136 dimensions into such self-contained patterns. Each new basis vector then carries two tags: **which pattern-type** it is, and **which copy** (the same pattern-type usually shows up several times among the 136). This is the only real computation — and it is small, because only ~ 12 (or 16, or 48) operations are involved.

Step 4 — Copy each pattern across the rest of its family

Step 3 was done at just *one* k per family. The other members are reached with the “move-it” operations from Step 2, so you don’t redo the work — you **copy the patterns over**. Applying the operation that sends the representative to another family member carries a copy of the pattern to that k -point. A pattern that started at a single k now has **one piece sitting at each family member**, so its size grows:

$$(\text{size at the representative } d) \times (\text{family size } f) = \text{full size } f \times d.$$

For diamond (family sizes 1, 4, 3; representative pattern sizes 1, 2, 3):

$$\underbrace{1 \times \{1, 2, 3\}}_{\text{Family A}} = \{1, 2, 3\}, \quad \underbrace{4 \times \{1, 2\}}_{\text{Family B}} = \{4, 8\}, \quad \underbrace{3 \times \{1, 2\}}_{\text{Family C}} = \{3, 6\},$$

i.e. exactly the set of block sizes $\{1, 2, 3, 4, 6, 8\}$ that the full calculation produces.

Step 5 — Read off the labels, and see what you bought

Every new basis vector now carries **three tags**:

- **which big object** $i = (\text{family, pattern-type})$;
- **where inside it**, $a = (\text{which family-member } k) \times (\text{which slot in the size-}d \text{ pattern})$ — there are $m_i = f \times d$ slots;
- **which copy** c .

The transformation from (k, I) to (i, a, c) is exactly this relabeling. And in this basis:

- **a rotation/reflection** just shuffles the *slots* a inside one object — it never mixes different objects i , and never touches the copy c ;
- **a symmetric object like W** does the opposite: it leaves every slot alone (acts identically on all slots) and only *mixes the copies* c within an object.

So all of W 's content is a small “copy $\times$ copy” matrix, one per object — instead of a full 1088×1088 matrix, you keep only $\sum (\text{copies})^2$ numbers. (For diamond $2 \times 2 \times 2$ that is 3384 numbers instead of $\sim 1.5 \times 10^5$, about a $44\times$ reduction.)

The whole thing in one breath

1. Fourier gets you to (k, I) , using up the translations so k is a good label.
2. Rotations sort the momenta into **families**.
3. At one k per family, the fixed-in-place operations carve the interpolation points into small **patterns** (type + copy).
4. Copy each pattern across its family $\rightarrow$ **big objects** of size (family) $\times$ (pattern).
5. Relabel everything as **(object, slot, copy)**. Symmetries then shuffle slots, while symmetric operators only mix copies — so the operator's data collapses to tiny per-object matrices.

The one fact worth remembering: **a group operation moves the slots and leaves the copies alone; a symmetric operator leaves the slots alone and moves the copies**. They are mirror images, which is exactly why the symmetric operator's data lives entirely in the (few) copies.

Appendix: the structure of the transformation Q itself

Call Q the change of basis that carries (k, I) to the symmetry labels (i, a, c) . It is a square $N \times N$ unitary, $N = \sum_i m_i n_i$. Its structure, from coarse to fine:

(1) Block-diagonal in k . Every symmetry-adapted basis vector lives at a *single* momentum (Step 5), so

$$Q = \bigoplus_k Q^{(k)}, \quad n_k \text{ blocks, each } n_{\text{IP}} \times n_{\text{IP}}.$$

Equivalently, $Q_{(k,I),(i,a,c)} = 0$ unless the momentum stored in a equals k : Q never mixes different momenta.

(2) Only one block per family is independent. The k -blocks inside one family are not free: the block at $g \cdot k$ is the symmetry-image of the block at k (apply the “move-it” operation g — a grid permutation with phases — and relabel). So you only need **one block per family**; the rest are generated.

(3) Each independent block is itself block-diagonal over patterns. A representative block $Q^{(k_0)}$ is the “reorganize the interpolation points into patterns” map of Step 3. In its natural form it is block-diagonal over the interpolation-point orbits (under the operations that fix k_0), with each orbit-block of size at most the number of those fixed operations. (A block built naively from a single invariant operator instead comes out dense inside $Q^{(k_0)}$; the orbit-by-orbit construction is the sparse, structured one.)

(4) **Unique data.** Collecting (1)–(3), the genuinely independent content of Q is

$$\#\{\text{independent entries}\} = \sum_{\text{family reps}} \sum_{\text{orbits } o} |o|^2,$$

one small $|o| \times |o|$ unitary per orbit, over one representative per family. Crucially these numbers are **fixed by the crystal symmetry and the interpolation-point positions alone** — Q carries *no material data*. In practice one never stores Q densely; it is regenerated from the crystal, the interpolation-point set, and a few group generators.

How entries are related (summary). (i) zero unless the momentum matches; (ii) a non-representative k -block equals the group-image of its family representative; (iii) inside a representative block, entries across different orbits are zero, and within an orbit they are the little-group symmetry-adaptation coefficients (pure group theory).

The diamond $2 \times 2 \times 2$ example (48 operations)

Here $N = n_k n_{\text{IP}} = 8 \times 136 = 1088$. The successive structures shrink the entry count as:

structure exploited	complex entries
dense $(n_k n_{\text{IP}})^2$	1,183,744
block-diagonal in k $(n_k n_{\text{IP}}^2)$	147,968
one block per family (3×136^2)	55,488
orbit-block-diagonal (unique data)	6,800

The final 6,800 splits over the three family representatives as $\sum_o |o|^2$:

family rep	# fixed ops	interpolation-point orbit sizes	$\sum_o o ^2$
Γ $(0, 0, 0)$	48	48, 24, 24, 12, 12, 8, 8	3872
X $(0, 0, \frac{1}{2})$	12	six 12's, ten 6's, two 2's	1232
L $(0, \frac{1}{2}, \frac{1}{2})$	16	five 16's, six 8's, two 4's	1696
total			6800

So the whole transformation — naively a 1088×1088 matrix with ~ 1.2 million entries — is fixed by only about **6800** genuinely independent numbers ($\approx 174\times$ smaller), organized as a handful of small (≤ 48) unitaries over just 3 family representatives, and those numbers are pure symmetry/geometry rather than material data. In short: Q is **block-diagonal in k , its blocks are symmetry-copies across each family, and each block is a direct sum of little (≤ 48 -dimensional) pattern-unitaries.**